

Distributed Food Delivery (DiDi)

Section 1: Written Report Draft

1. System Overview & Purpose:

- The DiDi Distributed Food Delivery Platform is a multi-tier, distributed coordination system designed to manage real-time interactions among four key stakeholders: Customers, Restaurants/Merchants, Couriers/Driver, and the Central Dispatch System.

- The core objectives of the system are:

- Handling concurrent order placements, status transitions, and payments with consistency guarantees.
- Performing low-latency, geo-aware matching between customer orders and nearby active delivery drivers.
- Maintaining fault-tolerant, synchronized order lifecycle states [Created -> Accepted -> Preparing -> Dispatched] across distributed nodes without single point of failure.

2. Components & Entity Identification:

A: Client

- **Customer Client:**
 - + Initiates order requests.
 - + Queries restaurant menus.
 - + Makes payments.
 - + Listens for live updates regarding order preparation and courier location.
- **Merchant/Restaurant Client:**
 - + Receives order requests.
 - + Accepts or reject orders.
 - + Updates prep status.
 - + Notifies the system when meals are ready for pickup.
- **Coruier (Driver) Client:**
 - + Transmits periodic heartbeat/telemetry updates (GPS coordinates, availability status).
 - + Accepts or declines dispatch offers.
 - + Marks [pick-up / drop-off] completions.

B: Server (Backend Microservices):

- **API Gateway / Load Balancer:** Acts as the reverse proxy and sole public entry point. Manages client authentication, rate limiting (throttling), and dynamic request routing.

- **Order Management Service:** Manages the order lifecycle state machine, order persistence, and transactional validation.
- **Matching & Dispatching Service:** Executes spatial indexing algorithms to discover available couriers within a bounding radius and orchestrates driver-to-order assignment.
- **Notification / Push Service:** Maintains open bidirectional streams (e.g., WebSockets / gRPC streams) to push live status transitions to connected clients.

C: Peers:

- **Courier Peer-to-Peer (P2P) Overlay Mesh:** At the edge layer, nearby active courier nodes form localized P2P clusters. These overlays can disseminate hyper-local traffic conditions and share telemetry data via gossip protocols, reducing continuous polling pressure on the centralized backend.

D: Shared Resources:

- **Merchant Inventory / Menu Items:** Shared read-write state requiring strict concurrency control to prevent overselling.
- **Driver Availability State:** Mutually exclusive resource state where a courier can only be committed to one active delivery bundle at a time (can't be both).
- **Distributed Order State Log:** Globally agreed sequence of state transitions replicated across nodes [Created -> Accepted -> Preparing -> Dispatched].

3. System Model Assumptions:

A. System Organization: Hybrid Architecture (Client-Server & Distributed Services with Edge P2P Overlay).

- Front-facing clients communicate with the backend using a traditional Client-Server model over TCP-based application protocols (HTTP/REST, WebSockets, or gRPC).
- The backend is organized as a Distributed Microservices Architecture, allowing independent horizontal scaling and failure isolation.
- Edge couriers can establish decentralized peer connections for localized coordination.

B. Timing Assumption: Asynchronous Distributed System:

- No global physical clock synchronization is assumed due to unpredictable clock drift across heterogeneous devices.
- Network transmission delays and messaging processing times are unbounded.
- Timeouts and heartbeat mechanisms are employed for failure detection, and logical ordering will rely on Lamport Logical Timestamps or Vector Clocks in subsequent milestone to order events casually (e.g., verifying whether an order cancellation occurred before or after courier dispatch).

C. Failure Assumption:

1. Node Failures (Crash-Recovery Model):

- Nodes are subject to fail-stop or crash-recovery faults. When a service node crashes, it halts execution and may eventually recover.
- Upon recovery, nodes must reconstruct their state using persistent write-ahead log (WAL) or state snapshot synchronization.
- Internal backend services are assumed to be non-Byzantine (nodes do not exhibit malicious or arbitrary behavior). Client inputs are sanitized and authenticated at the API Gateway.

2. Link Failures:

- Network channels are considered unreliable: messages may be delayed, dropped, reordered, or duplicated. Network partitions may occur.
- Reliable transport (TCP) coupled with application-level acknowledgements (ACKs), exponential backoff retries, and Idempotency Keys are enforced to guarantee "at-least-once" delivery semantics without duplicate side effects.

3. Storage Assumptions:

- Each backend service has access to persistent, crash-resilient local or distributed storage. Volatile in-memory state is flushed to disk or replicated across a quorum ($N / 2 + 1$) of nodes before committing transactions.

4. Scalability, Dependability, and Resource Sharing:

A. Scalability:

- **Horizontal Scaling:** Core business services (Order Service, Dispatch Service) are designed to be stateless, allowing dynamic auto-scaling behind load balancer.
- **Spatial Sharding:** The driver and order dispatch spaces are partitioned geographically (e.g., using Geo-hashes or Uber H3 grid systems). Each geographical zone operates independently, limiting the blast radius and computational load.

B: Dependability:

- **Fault Isolation & Queuing:** If the Matching Service encounters downtime, incoming orders are safely enqueued in durable message queues by the Order Service, preventing order loss and ensuring eventual processing upon service recovery.
- **Health Probes:** Active liveness and readiness probes decouple failing instances from the load balancer.

C: Resource Sharing & Concurrency Control:

- **Concurrency Mechanism:** Such as Distributed Mutual Exclusion via locks or Optimistic Concurrency Control (using version checks) ensure that race conditions - such as two couriers accepting the same dispatch offer simultaneously - are strictly prevented.

5. Architecture Diagram:

